

Final Project Presentation

Task Manager App

Group 2

Alexander Anderson, Dario Coreas, Sebastian Linares

Project Overview

Purpose

- Create a task manager application which can help organize tasks and lead individuals to manage time more efficiently.

Audience

- Our target audience are college students because college students invest time into how they organize their assignments, extra-curricular activities, and other tasks.

Main Problem Solved

- Eliminates disorganization by maintaining visibility on active tasks and grants users access to a platform where task management is a simple, straight-forward process.

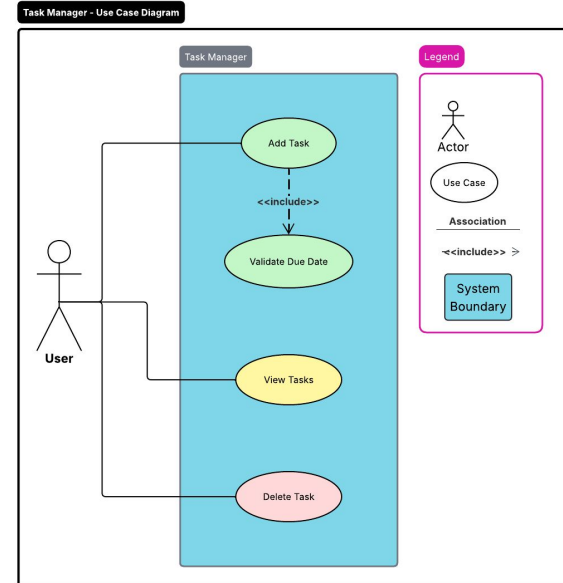
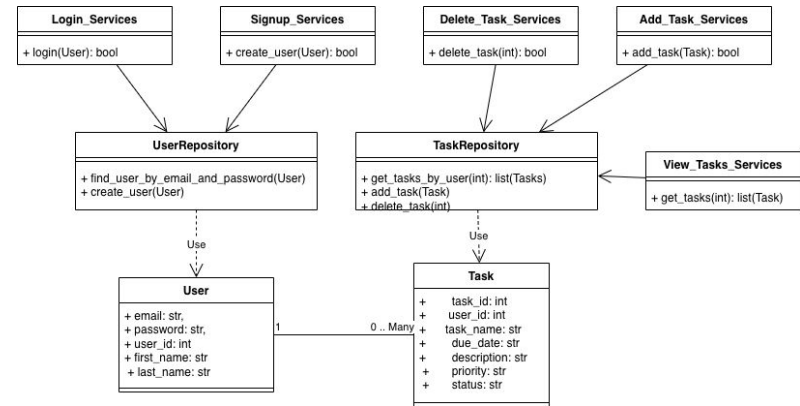
Development Process

Planning

- Team members met bi-weekly on Tuesdays and Thursdays to discuss project board tasks, provide updates on tasks, and ensure milestone requirements were completed.

Requirements

- The requirements our team was able to determine that would be necessary to have a functioning application that would meet the purpose of this project were:
 - Users need to have the ability to add a task.
 - Users need to have the ability to add a due date for their task.



Outstanding Features

Description Field

- Users can add descriptive text for their tasks which helps for specific reminders that might need notes or further details associated to a task.

Flask

- Acts as the core controller layer, managing route dispatching, request payloads, and task state transitions.

User Management

- Leverages SQLite to store and update user task data. Database maintains user information allowing access to stored active tasks.

Engineering Practices

Design Principles

- SOLID
 - Classes have one single responsibility, entities are open for extension but closed for modification, multiple interfaces used instead of one large general purpose interface, depends on abstraction.

Patterns

- Model-View-Controller Pattern
 - Model
 - Focus is on managing app data, updating databases, and processing business rules.
 - View
 - Graphical interface that user interacts with. Receives data through the controller.
 - Controller
 - The messenger between the model and the view.

Engineering Practices

Testing

- **Unit Test**

- **Verify individual components independently.**
- Example: **Login Service**
 - Verifies that a user can log in with valid credentials.
 - Rejects invalid usernames or passwords.
 - Uses mocked repositories to isolate the business logic.

- **Integration Test**

- Tests the interaction between the **Service**, **Repository**, and **SQLite database**
- Confirms that a created task is actually stored and can later be retrieved from the database.

- **Code Coverage**

- Measured test coverage using [coverage.py](#).
- Generated both terminal and HTML coverage reports.
- Used the report to identify untested code and improve test quality.

Demo

Future Enhancements

New Features

- Users should have the ability to associate a task to a college class to help further organize tasks. Opportunity to add an optional text field that will help users type in a class when creating a task.
- Users need to ability to prioritize tasks. There could be an option to color code tasks on the user dashboard such as highlighting a task red for high priority or yellow for medium priority.
- Possible Gantt Chart Project Shift
 - Ability to complete tasks, get the dopamine hit of checking the box! YAY accomplishment. Task will grey out and could possibly create a view of completed tasks
 - Add predecessor requirements. Tasks could have a optional predecessor task id that must be completed before the current task can be completed. Could possible add a view what tasks can be worked on right now.
 - Having an estimated time it takes to do the task, this coupled with the predecessor you could estimate total time a project would take.
 - Ability to break down big tasks into subtasks.
- Integrate Google Gemini to provide suggestive tips on how to complete task quicker.

Questions?